Computer Graphics: Lecture 1

Welcome to Real-time 3D!

Welcome!

You have done some cool things as a computer science major:

- You have learned and implemented a wide range of algorithms…
- Data structures too…
- You have developed user-facing applications (GUIs)

You have learned many languages, and built many projects, to do all sorts of cool things.

However, there is a set of capabilities inside your computer that you have not even *begun* to unlock. The computer is **way** more powerful than you've been able to take advantage of.

The computer's superpower

Almost every modern computer (including phones) contains a powerful **co-processor** called a GPU–a graphics processing unit. This GPU might be on a special expansion card, like this RTX 5090 here:


GPUs are incredibly powerful.
Clair Obscur: Expedition 33
They can produce beautiful 3D scenes like this one dozens or hundreds of times per second

And guess what?...

...You *aren't even using it.*

All of that power is just sitting there.

You have to explicitly write code for the GPU in order to use it.

No, your compiler will not optimize this for you. Programming for the GPU is very different from programming for the CPU, and it requires the code be architected in a particular way.

So let's stop having this powerful behemoth in your computer sit idle. Let's learn to *harness* it.

They can do more

Many of you are interested in making video games. That's okay, that was/is my motivation for learning computer science.

But many of you aren't. Guess what: **this class is still for you.**

GPUs are a powerful, versatile co-processor that is built into every modern phone, tablet, laptop, and desktop.

And it turns out, using the GPU to accelerate general computations is very similar to using it to draw cool scenes.

GPUs run shaders

A program that runs on the GPU is called a **shader**.

It was called that because, originally, these programs were used to visually shade 3D objects.

We will learn how to use them for that purpose, but they can also just run whatever computation you want on the GPU.

And there are so many useful computations to run…

Things GPUs are used for

- GPUs are what made the LLM revolution possible. The neural networks that power LLMs are glorified lists of matrix multiplications, which your GPU can do extremely quickly.[1]
- Physics simulations can be done on the GPU
- GUIs can be rendered on the GPU
- Complicated scientific processing can be done on the GPU
- Many algorithms can be made much faster with a GPU. Here is an implementation of Radix sort.
- And, last but not least, they can draw awesome 3D graphics.

[1] note: if you're not a fan of LLMs, don't blame your GPU. It just wanted to draw cool video game scenes.

What about engines?

You might wonder if we're going to learn one of the major 3D engines: Unreal, Unity, Godot, etc.

Actually, we're going to learn how they work.

In fact, we'll be building our own.

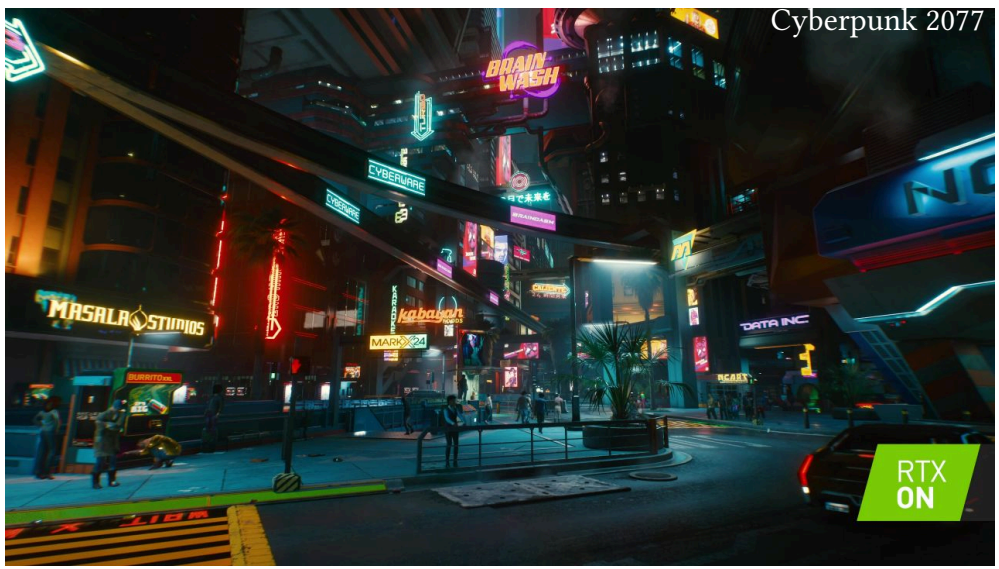
This might seem strange: why bother? There are some interesting reasons…

Reasons to make our own

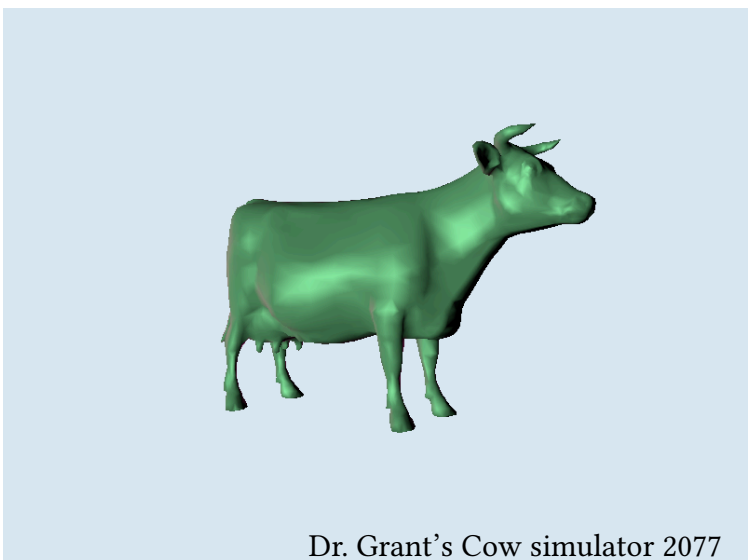
- 3D engines are highly customizable. To customize them you have to understand the basic 3D primitives. [Cool story: JPEG shader]
- These basics are the same regardless of engine: Matrices, Shaders, Meshes, these are always things you have to understand to get the most of your 3D engine. We will gain this engine-independent knowledge.
- You can do a lot more with GPUs than 3D graphics. It turns out learning to write programs for them unlocks tons of technologies (including AI).

Expectation management

Of course, I need to manage your expectations…



Expectations



Reality

[The big limiting factor for the left screenshot is artistic skill and time]

I hope you will get a lot out of this class

In this class, you will develop a simple realtime 3D graphics engine. In doing so, you will hopefully learn/gain a lot:

- How to interact with the GPU, including writing shaders. (this carries over to doing other things on the GPU besides graphics)
- Some really cool 3D math. My experience is that I understood the math way better after learning 3D graphics. Students have told me that this class makes it click[1]
- An awesome portfolio item: you'll make browser-based 3D apps.
- Last but not least: How your favorite game engines work.

[1] Don't worry, we review the math you're going to need as it comes up. I don't assume you remember very much from linear algebra.

A secret about me

This is my favorite class to teach.

I can’t resist talking about ancient 3D technology.

I will try my best not to bore you.

A lot of it is surprisingly relevant. And it’s also plain interesting.

Unreal Tournament: Map “Galleon”

Questions?

Let's talk about the syllabus

This is designed to be an informative but chill class.

There are no tests, only projects.

However, there are some nuances to how these will be graded.

Let's go over the syllabus together and make sure we understand everything…

Questions about the syllabus?

The chosen API

There is more than one way to interact with the GPU.

In general, we use an **API**, an "application programming interface" in order to run programs on the GPU.

There are many choices of API, but they are broadly split between

- Legacy APIs, like OpenGL and DirectX < 12. (We used to use OpenGL)
- Modern APIs, like Vulkan, Metal, DirectX 12, and WebGPU

The differences are pretty big…

Legacy vs. Modern

GPUs have changed a lot in the past 30 years.

It used to be they were a glorified triangle drawer. They could stretch the triangle, texture it, and even apply simple lighting or fog.

Those functions were built-in, and the APIs, such as OpenGL 1.x, had functions you could call to turn them off and on.


Spyro the Dragon (PS1)

Legacy vs. Modern (2)

Over time, GPUs became more and more flexible, to the point that they are full-on programmable co-processors.

At this point, legacy APIs became very clunky. You can still use them, but you end up dealing with a ton of jank. [examples]


Spyro the Dragon: Reignited

Modern graphics APIs

Modern APIs are designed to mirror the way that video cards actually work.

This means that it's a lot more predictable whether something will be fast or slow.

Because they are designed around the way video cards actually work, you need to personally know a bit more about how they work. Don't worry, that's on the agenda!

But which modern API will we learn?

We will be using WebGPU

My chosen API for this class will be WebGPU.

There are several huge benefits to using it:

- It runs on practically anything
- It works both in the browser or on the desktop.
- Great programming language support.
- Great debugging support: your browser will automatically alert you to lots of issues.
- Easy to get set up. A few dozen lines of code for your first program.
- Secretly uses Vulkan, Metal, or DX12 as a backend, but designed to be trivial to map to them (very little overhead).

Why not something else?

- Metal is well-regarded, but it only works on Apple platforms.
- Dx12 only works on Microsoft platforms.
- Vulkan runs on everything but, uhh…
- We used to use WebGL, but it's really showing its age. It's quite janky.

WebGPU is honestly a perfect fit for this class. It will work on any of your computers (as long as the browser is up to date–make sure to enable WebGPU support if you built your chromium from source or something).

WebGPU for the web

Despite the name, WebGPU works on desktop **or** web.

The Rust bindings are quite good if you want to experiment with modern desktop 3D on your own.

We will be using it on the web, however. This ensures that everyone can use the same code and it minimizes the platform-specific issues of getting set up.

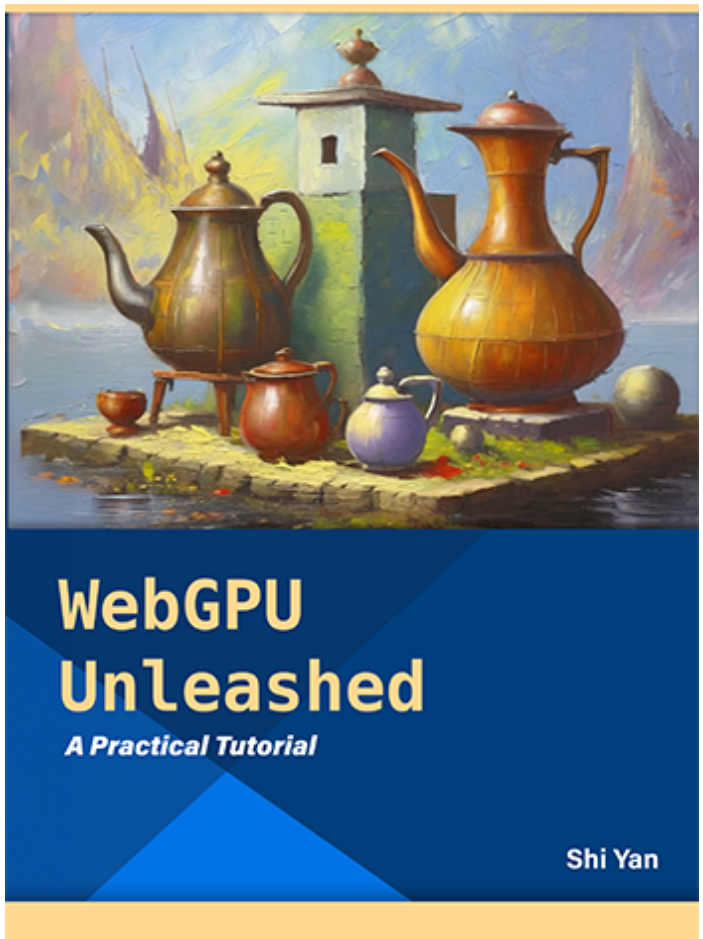
It also means that you'll get a killer portfolio item for your homepage.

The book

The main book we'll be using is WebGPU Unleashed by Shi Yan.

This is an awesome open educational resource with really cool samples that run inside the browser.

It's a great book, I was stoked to find it as I was designing this class.



The book (2)

The order of topics has been adjusted to flow mostly with the book, to make it a bit easier to get a “second coverage” of the material I introduce.

This is a bit different than my other classes where the book is rarely referred to. You still don’t strictly need it, but the way the class is organized should make it a bit easier to get extra help from the book.

And of course, there’s a new possibility due to the progress of technology…

AI policy

I have two minds about AI:

- It's an amazing learning tool. It can tutor you and address precise questions that you may not be able to find answers for.
- Debugging computer graphics applications can be hard. AI saved my bacon a few times when there were obscure layout or matrix issues.
- It can test you or reframe material until it clicks. This is huge.

On the other hand:

- It can give you a false sense of security about whether you understand something, causing you to not end up learning important concepts.
- We are engineers, not vibe-coders.

AI policy (2)

I won't be banning AI. You can use it in the class.

However, you *must* cite the code you write with it, and you must be prepared to explain every line you submit.

I'm serious about this. I'm very chill about letting you use it, as long as you follow these rules.

Why? Because it is very important to understand the degree to which you are relying on AI. My experience has been that people who use it don't realize how dependent they are on it, and it's often a shock.

Code reviews

Therefore, I am permitted to ask you for a code review: a small in person meeting in which I ask you what your code is doing.

I am looking for evidence that you do understand, not evidence that you don't, so these aren't designed to be scary/strenuous.

However, my experience is that people who overuse AI almost never cite it. Please don't make that mistake. Uncited code is more likely to result in a code review than cited code.

Citations

You don’t have to cite code you get from me or these slides.

Just leave a small comment at the top of the block of code with the source. Either a URL or just “LLM” or “AI”.

If your code is cited and the citations are < 50% of your code, it is very unlikely you will have any issues at all.

Working with friends

It would be strange if I permitted AI use but not working with friends.

My experience is that this class is awesome for working with friends. I've seen lots of students help each other over the finish line here, and I've seen very little outright cheating.

Therefore, the rules for other humans are the same as the rules for AI: please cite their contributions.

That's it. If you end up getting everything from someone else, that's not good because you still need to understand every line. However, feel free to help each other debug or offer suggestions.

Questions?

The Programming Language

To start using WebGPU, you can literally just type the code into a `<script>` block in your HTML.

However, we want to benefit from static typing. Trust me, it makes WebGPU development *dramatically* easier

This API loves using descriptor objects instead of normal method parameters. It is such a relief to just hit `shift+tab` and have all the required fields pop up.

Therefore, Typescript will be the main programming language.

Typescript

Don’t worry if you don’t know Typescript. We’ll be reviewing all the essential features as we need them.

The features we need will be pretty simple:

- Classes
- Values (objects, strings, numbers, etc.)
- Methods
- Fields
- Variables
- Loops
- Printing debug messages

Node

To write typescript, you need to have a package manager installed called “Node Package Manager” (NPM).[1]

Please download it here.

We’ll be using it next week, but you’ll want to hit the ground running.

[1] there are other node-like package managers such as bun. I don’t really care what you use, but you need to have node configurations so the TA only has to have node.

WGSL

Another programming language we'll be using is called WGSL.

This is the language used to write those shader programs that run on the GPU I was talking about.

It's a relatively new language, whose syntax was inspired by Typescript and Rust.

I will assume that you've never even heard of it, and we'll learn what we have to when it comes up.

A basic setup

Here is a simple setup that you can use that won't require too much work to learn.

Please go ahead and install these things this week so we can hit the ground running next week:

- VS Code as an IDE
- Install the `WGSL` and `WGSL Literal` packages for shader highlighting
- Install Node for your platform.
  (If you're on windows, Choclatey might be easier to use than Docker)
- We will set up a node package for your assignments next week.

That's it. Typescript language support should work out of the box.

Questions?

If time permits

If there's some time remaining in class, permit me a small question about your motivation.

What are some of the reasons people are taking this class?

- To learn how video games work (which games?)
- Interest in scientific simulation and visualization
- Wanting to harness the GPU
- Needed a systems elective (no shame in that–this is a good one!)

We can also look ahead at some of the topics we are going to cover. Which of these interest you?

Questions?